

Image depth estimation

Samuel Kuchta, Marek Bahník

1. Task definition

Our task was to design and implement an architecture incorporating **ResNet** encoder backbone for estimating depth information from monocular images. The proposed CNN should take as input a 2D color image, and output a depth map estimation with a resolution matching the input image.

Our objectives were specifically to design an architecture of CNN that can effectively learn and estimate depth information from monocular image. The design has been done using PyTorch and custom development framework created by Bc. Samuel Kuchta as a topic of his bachelors thesis¹. This project is contained in a GitHub repository², which has all the necessary files to perform the training, testing, validation and graph plotting and showing the output image. The repository contains a *README.md* file containing the instructions on how to use the project.

2. Existing solutions

There has been a lot of development in recent years regarding depth estimation from monocular images as well as other depth estimation tasks such as structure-from-motion tasks. Most of the [top performing papers regarding monocular depth estimation](#) use some sort of U-Net architecture of CNN as a base of the model. One study from 2022 discusses employing transformers to enhance the performance of pure CNNs[1]. Some for instance are trained in a self supervised manner[4]. For an in-depth comparison visit some of the recent method surveys[3][2].

3. Our solution

As stated in previous chapter, most architectures today are inspired by a U-Net[5] architecture and ours doesn't differ. The only major modification is that we do not use skip connections between the encoder

and decoder layers. The U-Net is comprised of an encoder and a decoder parts. The encoder has two main tasks: to extract features from the input image and to down-scale the image. On the contrary, the decoder up-scales the image and fuses the previously extracted features to create an altered version of the input image. In our case, the input is an RGB 2D image and output is the estimated single-channel depth map of the input image.

3.1 Work distribution

Work has been split between:

- **Samuel Kuchta** - CNN training framework modification, design of training and testing
- **Marek Bahník** - model architecture, training and documentation

3.2 Tools

During the development of this model, we have used several tools, which helped us with simplifying the process and let us focus mainly on the important parts of the project. This model has been made using Python and PyTorch. Using GPU can speed up the calculations and therefore CUDA tools have also been used when training the model. Last but not least, the already mentioned framework developed by Bc. Kuchta has been used to ease the training process and detecting problems.

3.3 Architecture

The architecture is mostly based on the U-Net architecture containing encoder, bridge and decoder. The general architecture can be seen on figure 1.

3.3.1 Encoder

As an encoder, we have chosen to use a pretrained model. The **ResNet** was the chosen architecture as it suits our needs. At first, we used the **ResNet18** to check, whether our network does function as intended. Later for main training we have switched to **ResNet34** to achieve better results. For better results, using high parameter count **ResNet** can be used, but the training

¹Bachelors thesis link

²Said GitHub repository.

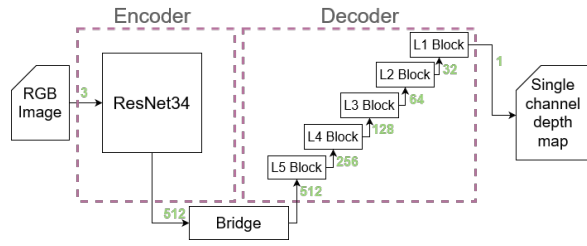


Figure 1. Full model architecture following the U-Net architecture. Numbers in green are amount of channels passing through the path.

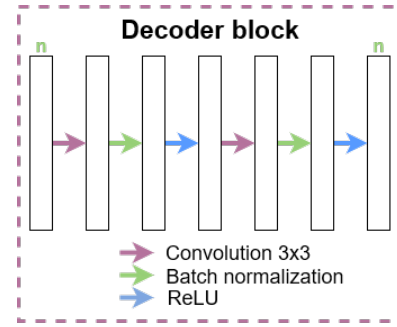


Figure 3. Design of every layer in the decoder.

process would be lengthier and therefore we chose to stick with smaller layered **ResNet** models. The weights have been pretrained on **IMAGENET1K_V1** dataset, which are provided by the **ResNet** package.

3.3.2 Bridge

The bridge is used to pass down features extracted by the encoder to the decoder in a finer scale. It functions as a connection between these two architecture parts. Our bridge consists of two decoder blocks as described in figure 3.

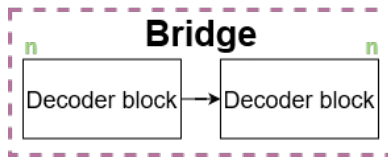


Figure 2. Bridge architecture containing two convolution blocks in image 3.

3.3.3 Decoder

We have chosen to create our own decoder network using the *torchvision nn* package and some custom code.

The decoder is a sequential network of connected blocks. These blocks pass up the extracted features from the bridge, upscale it and fuse the features to create a single channel output tensor describing the depth estimation of the input image. The decoder is split into layers. Every layer reduces the number of channels by half, except the last layer, that reduced 32 channels into one. There are 5 layers, as seen on figure 1. Each layer is called a **block**. These blocks all are defined by the same sequence of operations but differ only in channel counts. An example of a block can be seen on figure 3. Every block consists of several operation in sequence. First operation performs a 3x3 **convolution** on the given tensor with a padding to maintain the input size. After the convolution, **batch normalization** follows and lastly **ReLU** as a non-linear function is applied. This set of operations is performed twice. The channel count does not change inside each block. The upscaling takes place in between each block.

3.4 Training

For training we have chosen to use the **CityScapes** dataset. This dataset contains 5000 annotated images taken by two cameras placed onto a car. The ground truth has been created from stereoscopic images as some fragments of dual camera image processing can be seen. In retrospective, different dataset might have been a better choice, i.e., **KiTTi** or **NYU-Depth**. These 5000 images has been split into three parts: 3000 images used for training, 1500 used for testing and 500 images for validation. We have used MSE as a loss function and ADAM as an optimizer.

In total the model has been trained for 210 epochs. After some experiments, we have chosen to use **ResNet34** as our backbone. All the reference images shown in this section are obtained with figure 4 as input.



Figure 4. Reference image for visualizing results.

The first 7 epochs have been trained on the smallest (224px x 448px) images with batch size of 4. After checking the results, we have tried 3 epochs on a middle (448x896) images with result seen at figure 5. This has been followed by 3 epochs of full sized (896x1792) images. The result of this following training can be seen at figure 6. Now the process changed a little bit. The batch size has been changed to 1. For training image sizes, we have started alternating between small, medium and large to allow for more diverse capabilities regarding input image sizes. Also the validation dataset size has been reduced to 100 to speed up the validation. From now on, we also began logging the validation and test losses to plot them for visualization

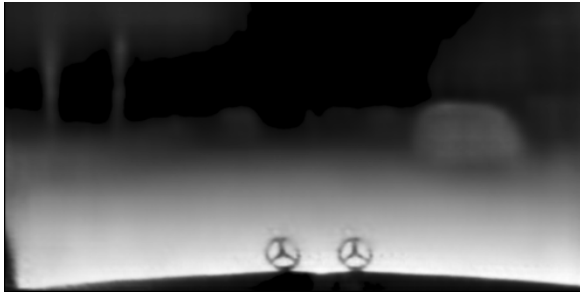


Figure 5. ResNet34 trained for 3 epochs on middle sized image after training as ResNet18 on small images.

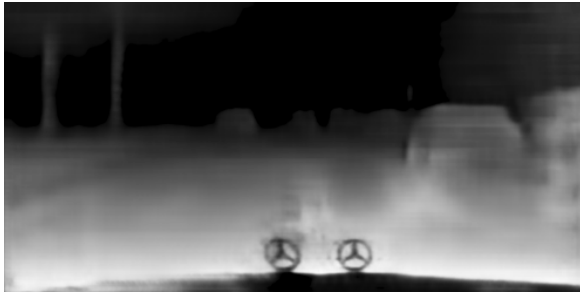


Figure 6. ResNet34 trained for 3 epochs on full sized image after training as ResNet18 on small images and 3 epochs of middle sized ResNet34 images.

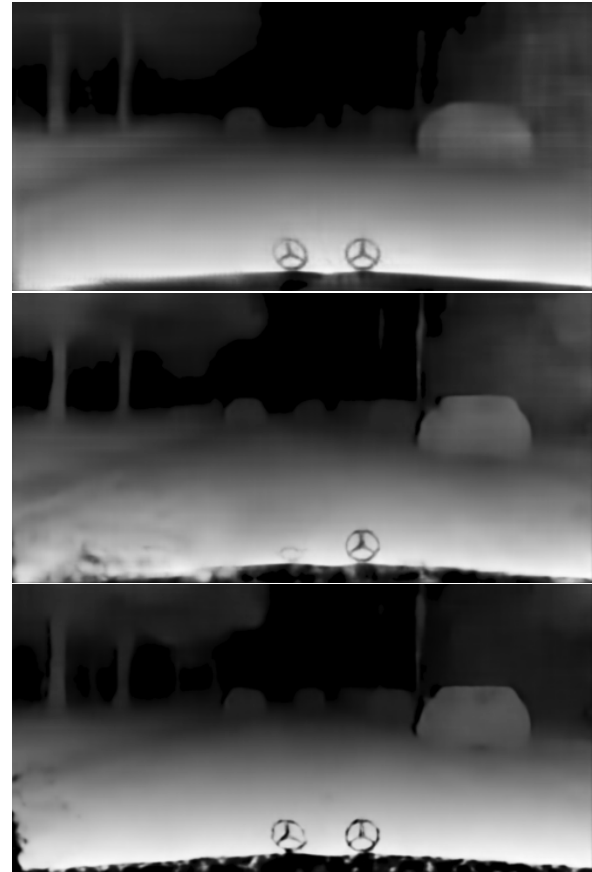


Figure 7. ResNet34 trained for 3, 53 and 100 epochs mixed image sizes tested on a full sized image.

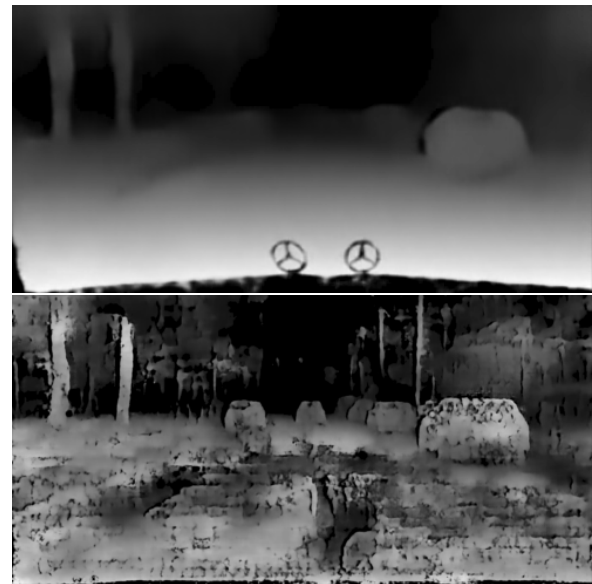


Figure 8. ResNet34 trained for 100 epochs mixed image sizes and 100 small images. Upper image is result of small input image, lower is full sized image input.

of training progress and status.

With this new setup, we have trained for 3 epochs to test, whether the proposed mix sized idea works. As this result has proven to be effective, we trained another 50 epochs. As seen on loss plot at figure 10, the network seems to be training well. From this point have switched to the full 500 images for validation and trained another 47 epochs to get to the full 100 epochs. The change in validation set size can be seen on figure 10 at epoch 53. After 100 epochs, the progress of the test results can be seen at figure 7.

As the loss has been still decreasing with more epochs, we have chosen to train another 100. Sadly we have left an error in the code and trained only on smallest images. This can be seen as well on figure 10 at the 100th epoch as a dip in validation loss. Resulting images after these another 100 failed epochs can be seen on figure 8. The small image performs very well, but the large one is highly fragmented. Still, the scene understanding and object recognition can be seen so more training would remove this mistake.

After this error occurred, we had to train 10 more epochs to balance the image sizes back. We were confident, that this was nothing permanent and as seen on figure 9, the test output of full sized image looks again very good.

The main process of training the ResNet34 has been logged and the plot of validation and testing loss can be seen on figure 10.

3.5 Evaluation

The evaluation has been done after every epoch. Based on evaluation, more training followed or the training stops. To prevent one bad epoch to stop the training altogether, we calculate the average of last several

167

168

169

170

171

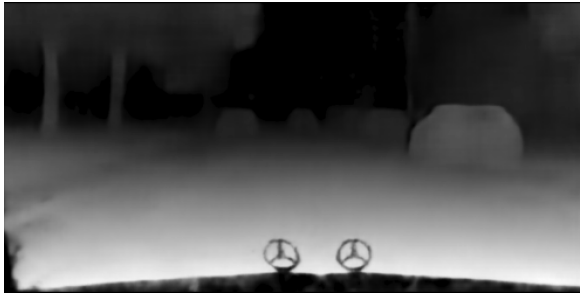


Figure 9. Output of **ResNet34** trained for 100 epochs mixed image sizes, 100 small images and 10 epoch mixed images correction.



Figure 10. Plot of full training loss. Dip at epoch 58 is a switch to full 500 validation images. Dip on epoch 100 is accidentally training on only small images. The increase in epoch 200 is correction of the previous error.

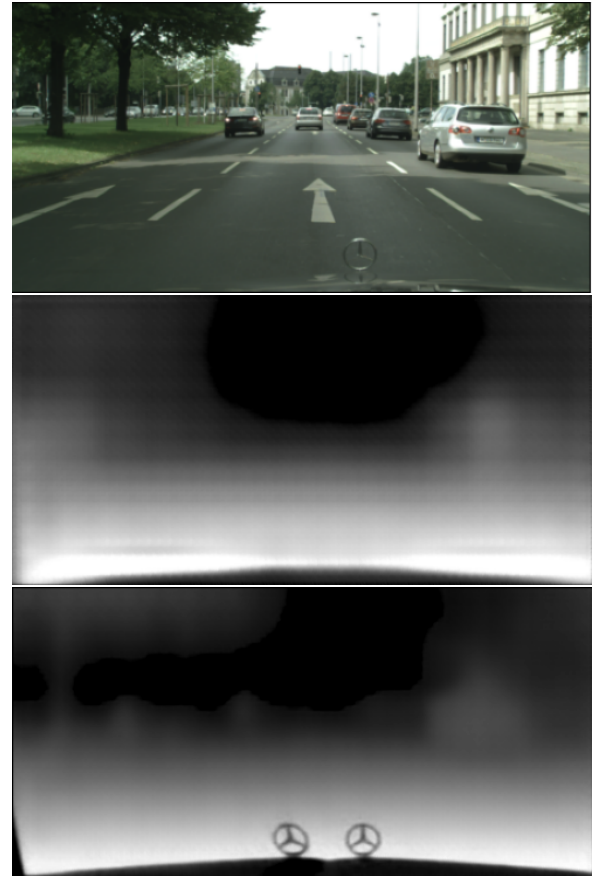


Figure 11. Results of performing depth estimation on the top most image with **ResNet** trained on 3 epochs with smallest images (middle) and 23 epochs with smallest images (bottom).

epochs and compare that to the current one.

4. Experiments

We had to experiment a little bit with the model architecture to have a functioning model, that processes the input image correctly. The main experiments have happened mostly during training. The image size itself was figured out by trial and error, as well as the thought of multiple different image sizes for training. The batch sizes have been tested during training as well and batch of one was the best solution based on the loss values.

On figure 11 you can see results of our experiment with smaller **ResNet18** backbone.

On figure 12 you can see output of the **ResNet34** model with only being trained on small images and having an input of middle sized image.

5. Results

The results are in our opinion satisfying the task definition. The model produces very believable and usable depth maps of a given image. Some results can be seen on figures 13, 14 and 15. To achieve better results, there are several steps we propose. First, the choice of the training dataset has not been great as

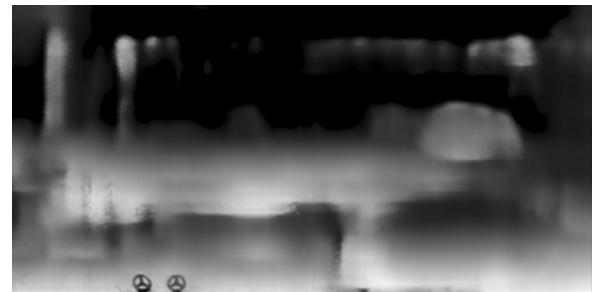


Figure 12. Result of inputting middle sized image to a **ResNet34** trained on only small images.

the dataset contains ground-truth images with a lots of fragments. This leads to our results containing them as well. Of course the error with training 100 epochs also makes the prediction not that precise, this would be fixed with having the network train for longer with correct setup. With these changes we believe the model would achieve better results.

6. Conclusions

We have attempted to create a CNN using **ResNet34** backbone and custom made decoder to perform a depth estimation on a monocular image. The model has been trained for approximately 200. The results are



Figure 13. Example result on the test dataset with full sized images.

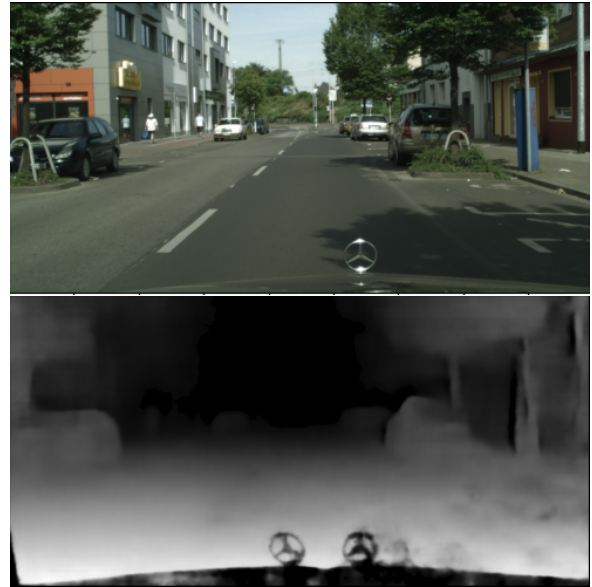


Figure 15. Example result on the test dataset with full sized images.



Figure 14. Example result on the test dataset with full sized images.

tal Signal Processing. Elsevier. 2022, vol. 123, p. 103441.

- [3] MING, Y., MENG, X., FAN, C. and YU, H. Deep learning for monocular depth estimation: A review. *Neurocomputing.* Elsevier. 2021, vol. 438, p. 14–33.
- [4] WANG, Y., LIANG, Y., XU, H., JIAO, S. and YU, H. SQLdepth: Generalizable self-supervised fine-structured monocular depth estimation. In: *Proceedings of the AAAI Conference on Artificial Intelligence.* 2024, p. 5713–5721.
- [5] YIN, X.-X., SUN, L., FU, Y., LU, R. and ZHANG, Y. U-net-based medical image segmentation. *Journal of healthcare engineering.* Hindawi Limited. 2022, vol. 2022.

satisfying, and the outputs are what is expected from model of this type. To enhance the capabilities of this model, we recommend training the model properly for up to 250 epochs and using dataset, that contains ground truth without any fragments.

References

- [1] LIU, Z., MAO, H., WU, C.-Y., FEICHTENHOFER, C., DARRELL, T. et al. A convnet for the 2020s. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition.* 2022, p. 11976–11986.
- [2] MERTAN, A., DUFF, D. J. and UNAL, G. Single image depth estimation: An overview. *Digi-*